

ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ПРОФЕССИОНАЛЬНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ИРКУТСКОЙ ОБЛАСТИ
«АНГАРСКИЙ ТЕХНИКУМ СТРОИТЕЛЬНЫХ ТЕХНОЛОГИЙ»

Курсовая работа

Тема: «Разработка web-приложения для размещения и запуска браузерных
игр»

Разработал _____ Фиузов Д. М.

Руководитель _____ Денисюк А. В.

Ангарск, 2025

СОДЕРЖАНИЕ

Введение.....	4
I. ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ	6
1.1 Браузерные игры	6
1.2 Web-приложение	8
1.3. Диаграмма «сущность-связь» базы данных	11
1.4. Выбор программного обеспечения и средств разработки	12
1.4.1. MySQL.....	12
1.4.2. DBeaver Community	13
1.4.3 Node.js и express.js.....	14
1.4.4 GitHub Desktop и VS Code.....	17
1.4.5 Движок EJS, Bootstrap 5 и Figma	19
1.5. Техническое задание	24
1.5.1. Общее описание функционала продукта	24
1.5.2 План разработки	26
1.6. Макет таблиц	26
1.6. Руководство по использованию приложения для разработчиков игр	29
1.6.1. Начало работы	29
1.6.2. Добавление игры на сайт.....	29
1.6.3. Форма редактирования игры.....	30
1.6.4. Удаление игры.....	31
1.7. Руководство конечного пользователя	31
II. РАЗРАБОТКА ПРОГРАММНОГО ПРОДУКТА	33
2.1. Архитектура проекта и файлов.....	33
2.2. Визуальная составляющая сайта	39
ЗАКЛЮЧЕНИЕ	42
СПИСОК ЛИТЕРАТУРЫ.....	43

						КР-09.02.07-Ф-30-23ПЗ					
Изм.	Кол.уч	Лист	№ док.	Подпись	Дата						
Разраб.	Фиузов Д.М.					«Разработка web-приложения для размещения и запуска браузерных игр»			Лит.	Лист	
Пров.	Денисюк А.В..									3	44
Н.контр											
Утв.						ГАПОУ ИО АТСТ					

Введение

В последние годы наблюдается заметный рост интереса к браузерным играм, что, в свою очередь, приводит к потребностям в создании удобных платформ для их размещения и запуска. Браузерные игры характеризуются простотой доступа: не требуют установки игр на локальные устройства и могут быть запущены непосредственно в веб-браузере. Это делает их особенно привлекательными для пользователей, стремящихся к быстрой игре без лишних хлопот.

Клиентское приложение будет включать в себя следующие функции:

- Возможность регистрации и авторизации на ресурсе
- Непосредственно игра в любую из доступных игр
- Добавление и редактирование своих игр
- Получения доступных достижений на ресурсе

Готовое решение позволит создать полноценную информационную систему для любой медицинской организации, которая будет совместима с программами других компаний и сможет быть легко масштабирована под запросы конкретной организации с минимальными доработками.

Актуальность темы

Актуальность разработки web-приложения для размещения и запуска браузерных игр обусловлена рядом факторов:

Рост популярности браузерных игр: с каждым годом увеличивается количество пользователей, играющих в браузерные игры. Они предлагают множество жанров, от аркад до стратегий, и могут привлекать игроков всех возрастов.

Независимость от платформы: Браузерные игры доступны на различных устройствах, что делает их универсальными. Пользователи могут играть на компьютерах, планшетах и даже смартфонах, не задумываясь о совместимости.

						КР-09.02.07-Ф-30-25ПЗ	Лист
Изм.	Кол.уч.	Лист	№ докум.	Подпись	Дата		4

Процесс создания: Многие разработчики стремятся размещать свои игры на различных платформах. Создание сайта, на котором можно не только проводить игры, но и делиться своими разработками, будет способствовать их популяризации и доступности.

Целью работы является Разработка web-приложения для размещения и запуска браузерных игр, включая возможность добавления пользовательских игр.

Для реализации этой цели были поставлены следующие задачи:

1. Изучить существующие платформы и их функциональные возможности.
2. Определить требования к функционалу и дизайну web-приложения.
3. Разработать архитектуру приложения и его интерфейс.
4. Реализовать систему регистрации пользователей и авторизации.
5. Обеспечить возможность загрузки и запуска игр.
6. Провести тестирование приложения и устранить выявленные ошибки.

						КР-09.02.07-Ф-30-25ПЗ	Лист
							5
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

І. ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

1.1 Браузерные игры

Браузерные игры — это видеоигры, которые могут быть запущены в веб-браузере без необходимости установки дополнительного программного обеспечения. Они становятся все более популярными благодаря своей доступности и удобству, позволяя игрокам наслаждаться игровым процессом в любом месте и в любое время, где есть доступ к интернету.

История развития

Ранние годы

Первоначально браузерные игры начали появляться в конце 90-х годов с развитием технологий, таких как Java и Flash. Эти технологии позволяли создавать интерактивные и графически насыщенные игры. Одним из первых успешных примеров является игра Neopets, которая была запущена в 1999 году и быстро приобрела популярность среди молодежи.

Переход к HTML5

С началом 2000-х годов Flash стал доминирующим стандартом для браузерных игр, однако его использование стало менее практичным из-за проблем с безопасностью и необходимости установки плагинов. С выходом HTML5 и CSS3 в начале 2010-х годов произошел значительный сдвиг. HTML5 позволил создавать игры, которые работают на различных устройствах, включая мобильные телефоны и планшеты, без необходимости установки дополнительных плагинов.

Особенности браузерных игр

Доступность

Одной из основных причин популярности браузерных игр является их доступность. Игроки могут начать играть в любую игру всего за несколько кликов. Это особенно привлекает людей, которые не хотят тратить время на загрузку и установку игр на свои устройства.

						КР-09.02.07-Ф-30-25ПЗ	Лист
							6
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

Социальный аспект

Браузерные игры часто имеют сильный социальный аспект, включая возможность взаимодействия с другими игроками через чаты, конкурсы и совместное выполнение заданий. Это создает сообщество и способствует большему вовлечению игроков.

Разнообразие жанров

Существует множество жанров браузерных игр: от аркад и головоломок до ролевых и стратегий. Это разнообразие позволяет охватить широкую аудиторию и удовлетворить различные предпочтения игроков.

Преимущества и недостатки

Преимущества

- Удобство: играть можно на любом устройстве с доступом в интернет.
- Бесплатность: Многие браузерные игры доступны бесплатно, с возможностью покупки внутриигровых предметов.
- Сообщество: Большинство игр имеют активные сообщества, что способствует социальному взаимодействию.

Недостатки

- Требования к интернет-соединению: Браузерные игры требуют постоянного доступа к интернету.
- Ограниченные возможности: по сравнению с настольными играми, браузерные игры часто имеют менее сложную графику и механику.
- Риск зависимости: Легкость доступа может привести к тому, что игроки проводят слишком много времени за игрой.

Будущее браузерных игр

С развитием технологий браузерные игры продолжают эволюционировать. Ожидается, что в ближайшие годы будут разработаны новые игры, использующие современные технологии, такие как виртуальная

						КР-09.02.07-Ф-30-25ПЗ	Лист
							7
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

и дополненная реальность. Также стоит отметить, что многообещающие перспективы могут быть связаны с внедрением блокчейн-технологий, что может открыть новые горизонты для монетизации и создания уникального игрового контента.

Классификация браузерных игр

Их можно условно разделить на два основных типа:

- Игры, использующие Flash и HTML5: до недавнего времени многие игры были созданы с использованием Adobe Flash. Однако с переходом к HTML5, браузерные игры стали более доступными и независимыми от плагинов.
- Многопользовательские онлайн-игры (ММО): позволяют игрокам взаимодействовать друг с другом в реальном времени в виртуальной среде.

1.2 Web-приложение

Веб-приложения — это программы, которые можно использовать через веб-браузер. Они отличаются от традиционных настольных приложений тем, что пользователям не требуется скачивать и устанавливать их на свои устройства. Web-приложения находят широкое применение в различных областях: от бизнеса до образования, от развлечений до социальных сетей, благодаря своей доступности и простоте использования.

История веб-приложений

Ранние этапы

В 1990-х годах, с появлением первых веб-браузеров, начались эксперименты с веб-приложениями, хотя они в основном ограничивались простыми формами и статическим контентом. Постепенно с развитием технологий веб-разработки, таких как CGI (Common Gateway Interface) и JavaScript, возможности веб-приложений начали расширяться.

						КР-09.02.07-Ф-30-25ПЗ	Лист
							8
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

Эволюция технологий

С начала 2000-х годов веб-приложения начали активно расти в популярности с выходом таких технологий, как AJAX (Asynchronous JavaScript and XML), которые позволили создавать более интерактивные и отзывчивые интерфейсы. Это привело к рождению таких популярных приложений, как Gmail и Google Maps, которые демонстрировали все преимущества веб-технологий. Совсем недавно наметилась новая эра веб-приложений, основанная на использовании фреймворков, таких как React, Angular и Vue.js, которые значительно упростили разработку сложных приложений и улучшили пользовательский опыт.

Преимущества и недостатки веб-приложений

Преимущества:

1. Доступность

Одним из главных преимуществ веб-приложений является их доступность. Пользователи могут получить к ним доступ с любого устройства с интернет-соединением и веб-браузером. Это способствует масштабу и охватывает пользователей по всему миру.

2. Обновления и обслуживание

Обновления веб-приложений можно проводить централизованно, без необходимости вручную обновлять каждую копию программы на устройствах пользователей. Это экономит время и ресурсы, а также позволяет обеспечить всем пользователям доступ к последним версиям.

3. Кроссплатформенность

Веб-приложения работают на различных операционных системах и устройствах. Они независимы от платформы, что устраняет проблемы совместимости, которые часто возникают с настольными приложениями.

Недостатки:

1. Зависимость от Интернет-соединения

						КР-09.02.07-Ф-30-25ПЗ	Лист
							9
Изм.	Кол.уч.	Лист	№ докум.	Подпись	Дата		

Основным недостатком веб-приложений является необходимость постоянного доступа к интернету. В условиях медленного или нестабильного соединения пользовательский опыт может значительно ухудшаться.

2. Безопасность

Поскольку веб-приложения обычно работают с личными данными пользователей, безопасность становится критически важной. Атаки, такие как SQL-инъекции и межсайтовый скриптинг (XSS), могут угрожать сохранности данных.

3. Ограниченные возможности

Хотя веб-приложения достигли значительных успехов в производительности, у них все ещё есть ограничения по сравнению с нативными приложениями. Некоторые функции устройства или производительность могут быть недоступны или работать медленнее.

Будущее веб-приложений

Век современных технологий предоставляет множество перспектив для веб-приложений. С увеличением популярности облачных вычислений и мобильных устройств веб-приложения будут продолжать развиваться. Использование искусственного интеллекта и машинного обучения в веб-приложениях, например, личные помощники и рекомендации, станет новым этапом в их эволюции. К тому же, с ростом сетей 5G скорость соединения будет увеличиваться, что положительно скажется на производительности веб-приложений.

Также следует отметить внедрение Progressive Web Apps (PWA) — новых технологий, которые объединяют лучшие аспекты веб и мобильных приложений, создавая приложения, способные работать оффлайн и использовать некоторые функции устройства, такие как уведомления и доступ к камере.

						КР-09.02.07-Ф-30-25ПЗ	Лист
							10
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

1.3. Диаграмма «сущность-связь» базы данных

Чтобы систематизировать знания, полученные в результате анализа предметной области, и перейти непосредственно к разработке базы данных, нужно сначала создать условную ER-диаграмму базы данных (от англ. entity-relationship diagram – диаграмма «сущность-связь»). Данный вид условных диаграмм данных предполагает лишь описание сущностей и связей между ними, без указания данных, которые содержит каждая сущность.

Для создания диаграммы будет использоваться открытое программное обеспечение «DBeaver».

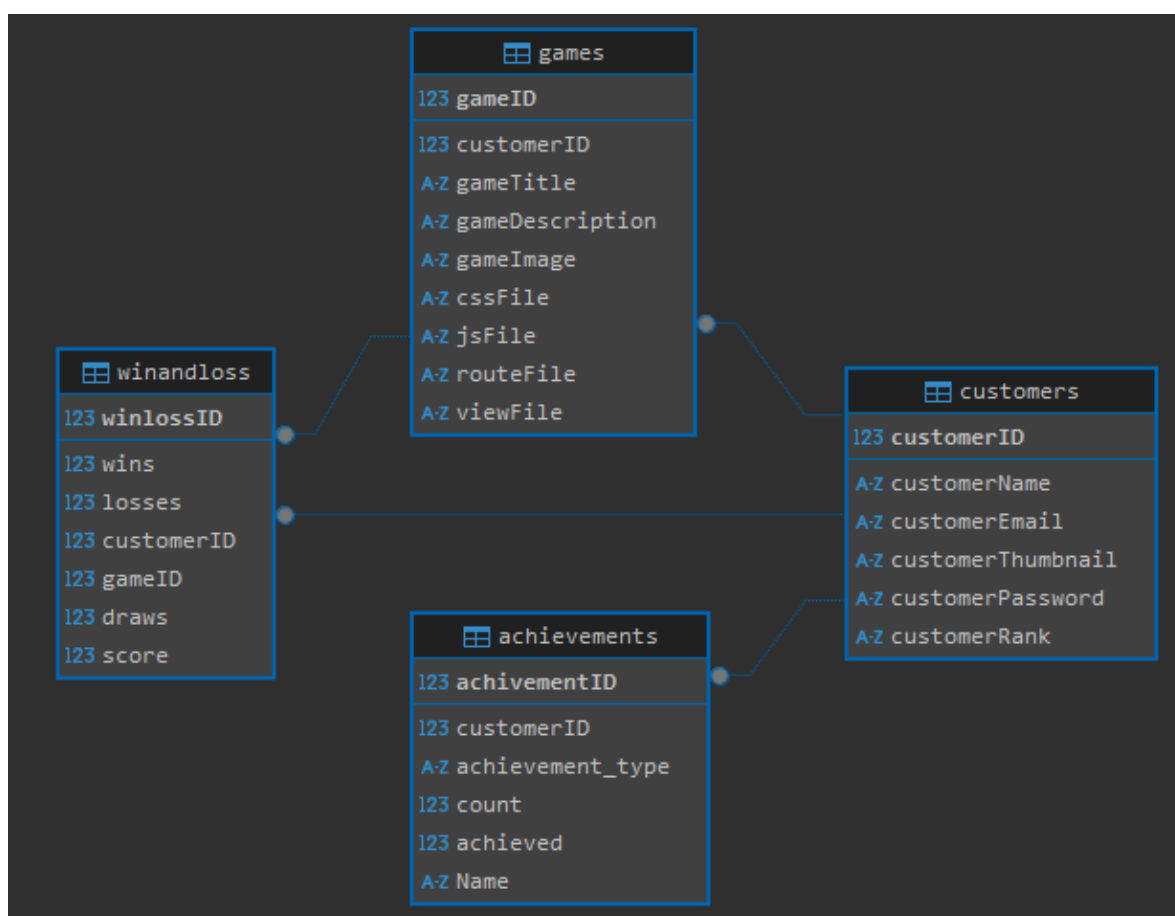


Рис. 1. ER-диаграмма базы данных

Данная диаграмма показывает, что:

- Каждый пользователь относится к нескольким Достижениям (т.е. каждый пользователь может получать разные достижения, что соответствует разным записям в таблице “achievements”)

- Каждый пользователь относится к таблице “games” где записываются данные обо все играх на сервере.
- Каждая запись в таблице “winandloss” относится как к таблице “customers“, так и к таблице “games”, что соответственно создает уникальную связь “gameID” <-> “customerID”, которая гарантирует что у конкретного пользователя в конкретной игре будет определенный счет.
- Каждый ключ имеет свойство «Каскад», это означает что при удалении привязанного к ней поля primary key все соответствующие ему записи в других таблицах будут удалены.

1.4. Выбор программного обеспечения и средств разработки

1.4.1. MySQL

Исходя из ER-диаграммы сущностей, можно выбрать программное обеспечение для создания базы данных.

Системы управления базами данных (СУБД) являются ключевыми компонентами современных приложений и сервисов, обрабатывающих большие объемы данных. В этом контексте MySQL занимает одно из ведущих мест среди реляционных СУБД, обеспечивая надежность, производительность и простоту использования.

1. История и развитие

MySQL была разработана в 1995 году шведской компанией MySQL AB и быстро завоевала популярность благодаря своей скорости и эффективности. В 2008 году MySQL была приобретена компанией Sun Microsystems, которая в свою очередь была куплена Oracle Corporation в 2010 году. Под управлением Oracle MySQL продолжает активно развиваться и обновляться, внедряя новые функции и улучшения.

						КР-09.02.07-Ф-30-25ПЗ	Лист
							12
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

2. Архитектура

MySQL использует реляционную модель для организации данных, что позволяет эффективно хранить и управлять ими. Основные компоненты MySQL включают:

- Сервер MySQL: обеспечивает выполнение всех операций с базой данных.
- Клиент MySQL: использует SQL-запросы для взаимодействия с сервером.
- MySQL Connector: библиотеки для подключения к MySQL из различных языков программирования.

3. Преимущества

Производительность: MySQL обеспечивает быструю обработку запросов и высокую доступность.

Открытость: MySQL — это система с открытым исходным кодом, что позволяет разработчикам использовать и модифицировать её без ограничений.

Поддержка сообщества: Большое сообщество разработчиков и пользователей обеспечивает обширную документацию, готовые решения и сопроводительные ресурсы.

1.4.2. DBeaver Community

DBeaver — это бесплатный инструмент с открытым исходным кодом для управления базами данных. Он поддерживает множество СУБД, включая MySQL, PostgreSQL, SQLite и другие. Это делает DBeaver многофункциональным инструментом, позволяющим взаимодействовать с различными системами в едином интерфейсе.

Основные функции

Интуитивно понятный интерфейс: DBeaver имеет простой и удобный графический интерфейс, что облегчает работу с базами данных.

						КР-09.02.07-Ф-30-25ПЗ	Лист
							13
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

Поддержка SQL: Предоставляет мощный редактор SQL с подсветкой синтаксиса, авто-завершением и возможностью выполнения последовательностей запросов.

Инструменты для визуализации данных: DBeaver позволяет визуализировать данные в виде диаграмм и таблиц, что упрощает анализ и интерпретацию информации.

Экспорт и импорт данных: поддерживает разные форматы для импорта и экспорта данных, включая CSV, XLSX и SQL.

Управление метаданными: упрощает работу с таблицами, представлениями, процедурами и другими объектами баз данных.

Преимущества DBeaver

Кроссплатформенность: DBeaver работает на различных операционных системах (Windows, macOS, Linux), что делает его доступным для широкого круга пользователей.

Поддержка множества СУБД: Возможность работать с различными базами данных из одного интерфейса экономит время и усилия.

Открытый исходный код: Пользователи могут адаптировать и расширять функциональность DBeaver согласно своим требованиям.

1.4.3 Node.js и express.js

Node.js и Express.js представляют собой мощный инструментальный набор для разработки серверных приложений на языке JavaScript. Node.js является средой выполнения, позволяющей выполнять JavaScript на стороне сервера, в то время как Express.js является популярным веб-фреймворком, построенным на основе Node.js. В сочетании они предоставляют разработчикам гибкость и мощные возможности для создания масштабируемых и высокопроизводительных веб-приложений.

Node.js: Основные характеристики

1. Что такое Node.js?

						КР-09.02.07-Ф-30-25ПЗ	Лист
							14
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

Node.js — это среда выполнения на основе V8 JavaScript Engine от Google, которая позволяет запускать JavaScript вне браузера. Node.js использует неблокирующую, асинхронную модель ввода-вывода (I/O), что делает его высокопроизводительным и способным обрабатывать множество одновременных соединений с минимальными затратами ресурсов.

2. Архитектура Node.js

Node.js основывается на модели событийного цикла (event-driven architecture), что означает, что он может обрабатывать события и запросы без блокировки выполнения других операций. Это позволяет разработчикам создавать высоконагруженные и масштабируемые приложения, такие как веб-серверы и API, которые могут одновременно обрабатывать огромное количество запросов.

3. Преимущества Node.js

Высокая производительность: Неблокирующая архитектура и использование V8 делают Node.js быстрым и эффективным для обработки запросов.

Масштабируемость: Поддержка асинхронных операций и простая архитектура позволяют легко масштабировать приложения.

Единый стек технологий: Использование JavaScript на обеих сторонах (клиентской и серверной) упрощает разработку и сокращает количество используемых технологий.

Express.js: Основные характеристики

1. Обзор Express.js

Express.js — это минималистичный веб-фреймворк для Node.js, который предоставляет удобный интерфейс для создания веб-приложений и API. Он предлагает инструменты и функции для управления маршрутами, URL-обработкой, обработкой запросов и ответов, а также middleware.

2. Основные функции

						КР-09.02.07-Ф-30-25ПЗ	Лист
Изм.	Кол.уч.	Лист	№ докум.	Подпись	Дата		15

Маршрутизация: Express.js поддерживает мощную систему маршрутизации, позволяющую обрабатывать различные HTTP-запросы (GET, POST, PUT, DELETE) по разным URL.

Middleware: Express предоставляет возможность использовать промежуточное программное обеспечение (middleware) для обработки запросов, что позволяет добавлять дополнительные функции, такие как аутентификация, логирование и обработка ошибок.

Поддержка шаблонов: Express.js можно легко интегрировать с шаблонизаторами, такими как Pug, EJS и Handlebars, что позволяет динамически генерировать HTML-страницы.

3. Преимущества Express.js

Простота и удобство: Легкая архитектура и возможность быстрого создания приложений делают Express фаворитом среди разработчиков.

Гибкость: Express.js предоставляет возможность настраивать приложение по вашим требованиям, позволяя использовать только необходимые модули.

Сообщество и экосистема: Большое сообщество разработчиков обеспечивает множество готовых решений и библиотек для Express.

Применение Node.js и Express.js

1. Сценарии использования Node.js

Node.js идеально подходит для следующих типов приложений:

Веб-серверы: Node.js может быть использован для создания высокопроизводительных веб-серверов, способных обрабатывать тысячи запросов одновременно.

API и microservices: С помощью Node.js можно легко создавать RESTful API и микросервисы для обмена данными между клиентом и сервером.

Чат-приложения: благодаря асинхронной природе Node.js разработчики могут создавать чат-приложения в реальном времени.

						КР-09.02.07-Ф-30-25ПЗ	Лист
							16
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

2. Использование Express.js для создания API

Установка и настройка: Установка Express.js осуществляется через NPM, после чего разработчики могут создавать свои маршруты и интегрировать middleware.

Создание маршрутов: Express.js позволяет создать маршруты для обработки различных запросов.

Обработка данных: Express.js предоставляет возможности для работы с данными, загружаемыми из запросов, что упрощает создание API для работы с базами данных.

1.4.4 GitHub Desktop и VS Code

В современном мире разработки программного обеспечения важным аспектом является не только качество кода, но и удобство работы с инструментами. GitHub Desktop и Visual Studio Code являются двумя популярными инструментами, которые значительно упрощают процесс разработки. Использование этих инструментов позволяет разработчикам оптимизировать рабочий процесс, улучшить командную работу и повысить производительность.

GitHub Desktop: Основные характеристики

GitHub Desktop — это графический интерфейс для управления репозиториями Git и работы с сервисом GitHub. Он предназначен для упрощения взаимодействия с системами контроля версий, позволяя разработчикам сосредоточиться на коде, не отвлекаясь на команды командной строки.

Основные функции

- Управление репозиториями: GitHub Desktop позволяет легко клонировать репозитории, создавать новые ветки и управлять мержами без необходимости использовать командную строку.

						КР-09.02.07-Ф-30-25ПЗ	Лист
							17
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

- **Отслеживание изменений:** Приложение предоставляет визуальный интерфейс для просмотра изменений в файлах, добавлению и удалению файлов из отслеживания.
- **Синхронизация с GitHub:** Простое управление синхронизацией с удалёнными репозиториями на GitHub, позволяющее отправлять и получать изменения одним кликом.
- **Совместная работа:** Поддержка работы с pull request'ами позволяет легко открывать, просматривать и управлять запросами на изменения.

Использование GitHub Desktop при разработке делает процесс работы с Git более интуитивно понятным, особенно для новичков. Графический интерфейс значительно упрощает операции, которые в противном случае могли бы быть трудоемкими. Это особенно полезно для визуализации изменений в коде и управления ветками, что способствует улучшению командной работы.

Visual Studio Code: Основные характеристик

Visual Studio Code (VS Code) — это легкий и мощный редактор кода, разработанный Microsoft. Он поддерживает множество языков программирования и предоставляет обширные возможности для настройки.

Основные функции

Редактирование кода: VS Code включает подсветку синтаксиса, автозавершение и поддержку расширений, что делает процесс написания кода более эффективным.

Интеграция с Git: VS Code имеет встроенную поддержку Git, позволяя пользователям выполнять команды Git непосредственно из редактора.

Расширяемость: Marketplace VS Code предлагает множество расширений для поддержки различных языков, инструментов и фреймворков, что позволяет адаптировать редактор под личные потребности.

						КР-09.02.07-Ф-30-25ПЗ	Лист
							18
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

Отладка: Встроенные инструменты отладки позволяют разработчикам быстро находить и исправлять ошибки в коде.

VS Code объединяет в себе простоту и мощные функции, что делает его идеальным инструментом как для новичков, так и для опытных разработчиков. Благодаря интеграции с Git и поддержке расширений разработчики могут легко настроить рабочее пространство под свои нужды. Это позволяет существенно экономить время при разработке и тестировании программного обеспечения.

1.4.5 Движок EJS, Bootstrap 5 и Figma

EJS (Embedded JavaScript) — это простой и гибкий движок шаблонов для JavaScript, который позволяет вставлять динамический контент в HTML. Он часто используется в разработке веб-приложений на Node.js и Express.js, обеспечивая удобный способ генерации HTML-страниц с динамическими данными.

Основные возможности EJS

Встраивание JavaScript: EJS позволяет встраивать JavaScript-код прямо в HTML с помощью специального синтаксиса. Это дает возможность работать с данными, создавать условия и выполнять циклы непосредственно в шаблоне.

Легкость использования: EJS имеет простой и интуитивно понятный синтаксис, похожий на стандартный HTML. Это позволяет разработчикам быстро начать работу без необходимости изучения сложных концепций.

Поддержка шаблонов: EJS поддерживает возможность создания частичных шаблонов, что упрощает повторное использование кода. Это особенно полезно при работе с общими компонентами, такими как заголовки или подвал.

Логика и условия: В шаблонах EJS можно использовать условные операторы и циклы, что дает возможность динамически изменять содержимое страниц в зависимости от условий, заданных в серверном коде.

						КР-09.02.07-Ф-30-25ПЗ	Лист
							19
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

Применение EJS в проектах

EJS часто используется в сочетании с веб-фреймворками, такими как Express.js. Он позволяет создавать динамические веб-страницы, комбинируя статический HTML с данными, полученными с сервера. Например, данные из базы данных могут быть переданы в шаблон EJS, который затем отформатирует их и представит пользователю в виде веб-страницы.

Преимущества и недостатки

Преимущества:

- Простой и понятный синтаксис.
- Быстрая интеграция с Node.js и Express.
- Мощные возможности для динамической генерации контента.

Недостатки:

- Ограниченные возможности по сравнению с более сложными движками шаблонов (например, React или Vue.js).
- Может привести к перегрузке шаблона сложной логикой, что усложняет его поддержку.

Bootstrap — это популярный фреймворк для разработки адаптивных и мобильных веб-приложений. Он был создан компанией Twitter и стал одним из самых используемых инструментов в веб-разработке. Bootstrap предоставляет разработчикам готовые компоненты и стили, которые упрощают создание красивых и функциональных пользовательских интерфейсов.

Новшества в Bootstrap 5

Bootstrap 5 был выпущен с рядом значительных улучшений и изменений по сравнению с предыдущей версией, Bootstrap 4. Основные новшества включают:

Удаление jQuery: В Bootstrap 5 была полностью убрана зависимость от jQuery, что позволяет использовать чистый JavaScript. Это уменьшает вес

						КР-09.02.07-Ф-30-25ПЗ	Лист
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		20

библиотеки и улучшает производительность, поскольку современные браузеры могут выполнять нативный JavaScript быстрее.

Нативные компоненты: Многие компоненты, такие как модальные окна и уведомления, теперь имеют поддержку на основе нативного JavaScript, что делает их более производительными и менее зависимыми от сторонних библиотек.

Расширенный сеточный механизм: Bootstrap 5 вводит новые классы для более гибкой настройки сетки. Теперь разработчики могут использовать дополнительные параметры для управления размерами и отображением элементов на различных экранах.

Новая система форм: В Bootstrap 5 улучшена работа с формами. Стилизация форм стала более удобной и оптимизированной, позволяя легко создавать отзывчивые и интуитивно понятные интерфейсы ввода данных.

Тема по умолчанию: Новый стиль по умолчанию в Bootstrap 5 делает его визуально более современным, с плавными границами и улучшенной типографикой.

Преимущества использования Bootstrap 5

Адаптивность: Bootstrap предоставляет мощные инструменты для разработки адаптивных веб-сайтов, которые автоматически подстраиваются под различные размеры экранов и устройства.

Быстрая разработка: благодаря готовым компонентам и легко настраиваемым стилям, разработка становится гораздо более быстрой, что снижает количество повторного кода и ускоряет процесс создания прототипов.

Активное сообщество: Bootstrap имеет большое сообщество разработчиков и обширную документацию, что делает решение проблем и обучение проще.

						КР-09.02.07-Ф-30-25ПЗ	Лист
							21
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

Кроссбраузерная совместимость: Bootstrap разрабатывается с учетом поддержки всех современных браузеров, что позволяет избежать проблем совместимости.

Недостатки Bootstrap 5

Стандартный вид: Использование Bootstrap может привести к тому, что сайты, созданные с его помощью, будут выглядеть слишком похожими друг на друга, если разработчики не настроят стили и не добавят уникальные элементы.

Размер библиотеки: хотя в Bootstrap 5 уменьшили зависимость от jQuery, сама библиотека всё еще может быть достаточно большой для небольших проектов. Разработчикам может потребоваться настроить сборку, чтобы включить только необходимые компоненты.

Заключение

Bootstrap 5 представляет собой мощный инструмент для разработчиков, желающих создавать адаптивные и современные веб-приложения. Его легкость в использовании и гибкость позволяют разработчикам сосредоточиться на создании функционального и привлекательного дизайна, сокращая время на разработку и тестирование.

Figma — это мощный облачный инструмент для дизайна пользовательских интерфейсов (UI) и интерфейсов взаимодействия (UX). Он позволяет дизайнерам, разработчикам и командам работать совместно в реальном времени, обеспечивая динамическое создание и редактирование графических элементов, прототипов и макетов дизайна. Figma стала популярной благодаря своей доступности и возможностям для многопользовательской работы.

Ключевые функции Figma

Совместная работа в реальном времени: Figma позволяет нескольким пользователям одновременно работать над проектом. Изменения видны всем

						КР-09.02.07-Ф-30-25ПЗ	Лист
							22
Изм.	Кол.уч.	Лист	№ докум.	Подпись	Дата		

участникам в реальном времени, что значительно упрощает командную работу и обсуждения.

Облачное хранилище: Все проекты хранятся в облаке, что обеспечивает доступ к ним с любого устройства. Это позволяет дизайнерам работать из любого места и не беспокоиться о сохранении файлов на локальном компьютере.

Компоненты и стили: Figma поддерживает создание компонентов и использование общих стилей, что упрощает повторное использование элементов дизайна и обеспечивает единообразие в проекте.

Прототипирование: Figma предоставляет инструменты для создания интерактивных прототипов, которые могут смоделировать пользовательский опыт. Дизайнеры могут связывать разные экраны и добавлять анимации, чтобы продемонстрировать, как будет работать интерфейс.

Плагины и интеграции: Figma поддерживает использование различных плагинов, которые расширяют функциональность инструмента. Существуют плагины для автоматизации задач, генерации контента, работы с данными и многие другие.

Преимущества использования Figma

Доступность: Figma является веб-приложением, поэтому пользователи могут получить доступ к проектам из любого места с доступом в Интернет. Это особенно полезно для удаленных команд.

Удобство интерфейса: Figma имеет интуитивно понятный интерфейс, который делает его доступным для пользователей с различным уровнем подготовки. Новички могут быстро начать работу, а опытные дизайнеры найдут все необходимые инструменты для создания сложных проектов.

Масштабируемость: Figma подходит как для небольших стартапов, так и для крупных компаний — она может обрабатывать большие проекты с сложными элементами дизайна.

Недостатки Figma

						КР-09.02.07-Ф-30-25ПЗ	Лист
							23
Изм.	Кол.уч.	Лист	№ докум.	Подпись	Дата		

Зависимость от Интернета: Поскольку Figma является облачным приложением, его функциональность ограничена без доступа к Интернету. В оффлайн-режиме работа с проектами невозможна.

Проблемы с производительностью: У некоторых пользователей возникают проблемы с производительностью при работе с большими и сложными проектами, особенно на менее мощных устройствах.

Платные функции: Несмотря на то, что Figma предлагает бесплатный тариф, некоторые продвинутые функции доступны только в платных версиях, что может стать преградой для небольших команд или индивидуальных разработчиков.

Заключение

Figma — это современный инструмент, который значительно облегчает процесс дизайна пользовательских интерфейсов и взаимодействия. Его функции совместной работы, доступность и простота использования делают его популярным выбором среди дизайнеров по всему миру. Figma продолжает развиваться и адаптироваться к потребностям пользователей, становясь незаменимым инструментом в сфере UI/UX-дизайна.

1.5. Техническое задание

1.5.1. Общее описание функционала продукта

Программный продукт будет выполнен в виде веб-приложения, разработанного с использованием Node.js и Express.js. Администрирование контента (размещение, редактирование и удаление игр) будет осуществляться через встроенный интерфейс администрирования, который обеспечивает удобное управление контентом. В дополнение к этому будет создан общедоступный клиентский интерфейс, позволяющий пользователям просматривать и запускать браузерные игры. Также будет система достижений.

Пользователи смогут:

						КР-09.02.07-Ф-30-25ПЗ	Лист
							24
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

- Просматривать список доступных игр на главной странице приложения.
- Запускать игры прямо из веб-интерфейса.
- Добавлять свои игры

Регистрация новых пользователей для получения доступа к дополнительным функциям, таким как сохранение и достижения.

Требования к программной среде

Готовый программный продукт должен работать на платформе, основанной на операционной системе с ядром Windows 10. Предпочтительно использование версий с долгосрочной поддержкой (LTS). Важно, что все операционные системы на базе Linux имеют открытый исходный код и свободную лицензию, что обеспечивает бесплатное использование без региональных ограничений.

Для корректного функционирования серверной части приложения потребуется установленная платформа Node.js версии 16 и фреймворк Express.js.

Для сохранения данных в сессию и последующего их использования будет использоваться дополнительный фреймворк express-session

Требования к аппаратной среде

Так как основная платформа проекта основана на операционной системе Windows, аппаратные требования не являются строгими. Рабочая машина может иметь любую архитектуру процессора, включая AMD x86-64 и ARM64, при условии достаточно системных ресурсов для запуска любого браузера.

Системные требования для пользователя

Взаимодействие пользователей с веб-приложением будет осуществляться через современные веб-браузеры, такие как:

- Google Chrome (версия 39 и выше)
- Mozilla Firefox (версии от Chromium 39 и выше)

						КР-09.02.07-Ф-30-25ПЗ	Лист
							25
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

1.5.2 План разработки

Разработка программного продукта планируется в срок до 1 месяца.
Процесс разработки будет включать в себя:

- Развертывание тестового стенда с СУБД MySQL, DBeaver на локальном компьютере для проектирования базы данных и настройки системы управления контентом;
- Разработку дизайна интерфейса общедоступного клиентского веб-приложения с помощью Figma;
- Разработку клиентского веб-приложения(frontend и backend);
- Тестирование программного продукта на локальном компьютере;

1.6. Макет таблиц

Теперь, когда было определено, на основе какого программного обеспечения будет разработана база данных и известно, какие существуют ограничения, можно определиться с физической структурой данных. Ниже приведена структура таблиц базы данных в том виде, в котором она будет объявлена в системе управления базами данных.

Таблица 1. Структура таблицы «customers»

Название поля	Тип данных	Пример данных
Идентификатор (customerID)	Int, primary key	-
Никнейм (costomerName)	Varchar (45)	«Владимир»; «Иван»; «Николай»; «Рупа»
Электронная почта (customerEmail)	Varchar (45)	«example@mail.ru»; «example@gmail.com»; «example@yandex.com»
Аватарка (customerThumbnail)	Varchar (255)	«/images/Thumbnail_1.png»; «/images/Thumbnail_2.png»; «/images/Thumbnail_3.png»;

Пароль (customerPassword)	Varchar (100)	(Хэшированное, т.е. зашифрованное значение пароля)
Ранг (customerRank)	Varchar (100)	«Разработчик»

Таблица 2. Структура таблицы «games»

Название поля	Тип данных	Пример данных
Идентификатор (gameID)	Int, primary key	-
Идентификатор пользователя (customerID)	Int, связано с таблицей customers	-
Название игры (gameTitle)	Varchar (255)	«Игра1»; «Игра2»;
Описание игры (gameDiscriptoin)	Varchar (255)	«Описание1»; «Описание2»;
Изображение игры (gameImage)	Varchar (255)	«/2/images/image.png»; «/2/images/image1.png»;
Файлы стилей (cssFile)	Varchar (255)	«/2/styles/style.css»; «/2/styles/style2.css»;
Файлы скриптов (jsFile)	Varchar (255)	«/2/scripts/script.js»; «/2/scripts/script2.js»;
Файл маршрутизации (routeFile)	Varchar (255)	«/2/routes/route.js»; «/3/routes/route.js»;
Файлы ejs представлений (viewFile)	Varchar (255)	«/2/views/index.js»; «/2/views/view.js»;

Таблица 3. Структура таблицы «winandloss»

Название поля	Тип данных	Пример данных
Идентификатор (winlossID)	Int, primary key	-
Идентификатор пользователя (customerID)	Int, связано с таблицей customers	-
Идентификатор игры (gameID)	Int, связано с таблицей games	-
Победы (wins)	int	«1»; «5»;
Поражения (loss)	int	«2»; «4»;
Ничьи (draws)	int	«2»; «6»;
Счет (score)	int	«200»; «2560»;

Таблица 4. Структура таблицы «achievements»

Название поля	Тип данных	Пример данных
Идентификатор (achievementsID)	Int, primary key	-
Идентификатор пользователя (customerID)	Int, связано с таблицей customers	-
Тип достижения (achievements_type)	Varchar (255)	«wins»; «losses»;
Количество (count)	int	«2»; «15»; «3»

Достигнуто или нет (achieved)	bool	«1»; «0»;
Название достижения (Name)	Varchar (100)	«Победы 1»; «Поражения 3»;

1.6. Руководство по использованию приложения для разработчиков игр

1.6.1. Начало работы

Для начала работы с веб-приложением для добавления игры необходимо открыть в веб-браузере на базе Chromium (напр. Google Chrome, Microsoft Edge) специальную ссылку для установки приложения на свое устройство и нажать на кнопку «Установить». После этого откроется веб-приложение и запросит ввод учетных данных (имя пользователя и пароль). После успешного входа надо перейти на страницу аккаунта и нажать кнопку мои игры.

1.6.2. Добавление игры на сайт

Для того, чтобы добавить новую игру, необходимо на странице «мои игры» необходимо нажать на пустую карточку с «плюсом»

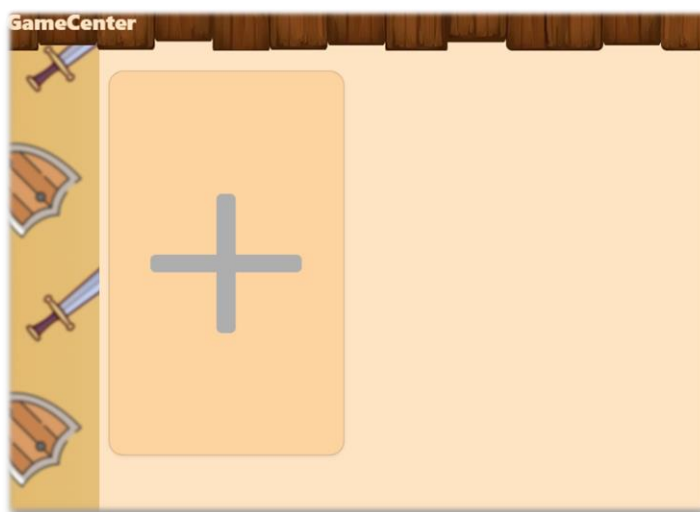


Рис. 2. Добавление игры

Затем заполнить форму соответствуя инструкциям, расположенным справа от полей для заполнения. Важно заполнять поля четко соответствуя инструкциям, чтобы не допустить ошибок в работе добавляемой игры.

Рис. 3. Форма добавления игры

После заполнения всех необходимых данных можно будет нажать кнопку добавить. После сохранения приложение осуществит переход обратно на главную страницу. Если приложение обнаружит ошибку в заполненных данных (например, не заполнено обязательное поле), вверху формы будет выведена подробная информация об ошибке.

1.6.3. Форма редактирования игры

Чтобы открыть форму редактирования добавленной игры необходимо снова перейти на страницу своего аккаунта и поле чего нажать кнопку «мои игры», в открытой вкладке выбрать игру которую хотите редактировать и нажать на ней «Изменить», после чего откроется форма редактирования, которая похожа на форму добавления игры за исключением того что в форме изменения видно какие файлы сейчас «привязаны» к этой игре и есть возможность заменить их на новые.

После введения новых данных и замены старых файлов на новые необходимо сохранить изменения нажав кнопку «изменить», находящуюся снизу формы редактирования игры.

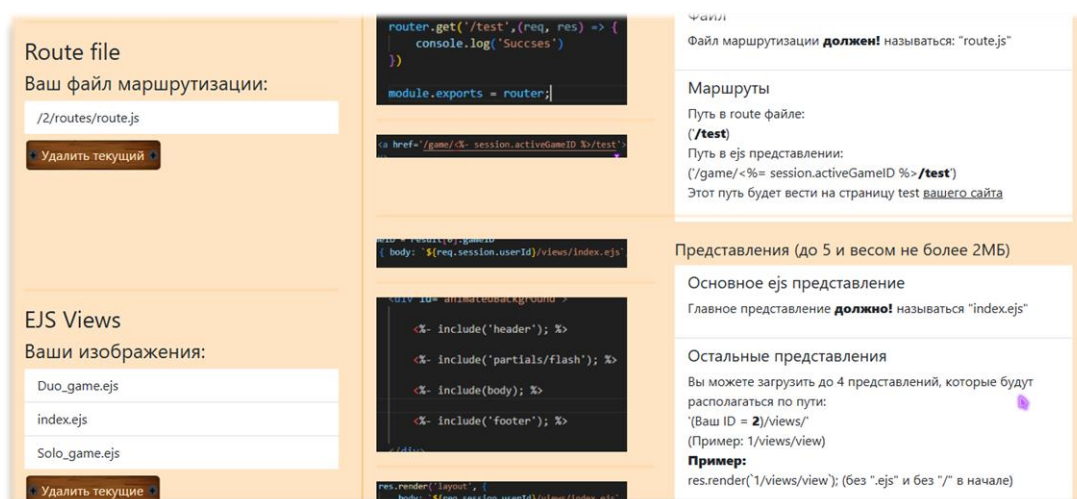


Рис. 4. Форма редактирования игры

1.6.4. Удаление игры

Для того, чтобы удалить игру из базы данных и с сайта, нужно открыть форму редактирования этой игры (см. п. 1.6.3), а затем нажать на кнопку удаления снизу формы редактирования, слева от кнопки изменения.

1.7. Руководство конечного пользователя

Для использования приложения сточки зрения обычного пользователя чтобы так сказать «просто поиграть в игры». Надо зайти в любой браузер (Google Chrome) и перейти на сайт. Далее необходимо лишь зарегистрироваться и войти в свой аккаунт. Далее достаточно выбрать понравившуюся игру и нажать «играть» после чего будет осуществлен переход на страницу этой игры, где можно непосредственно перейти к игре.

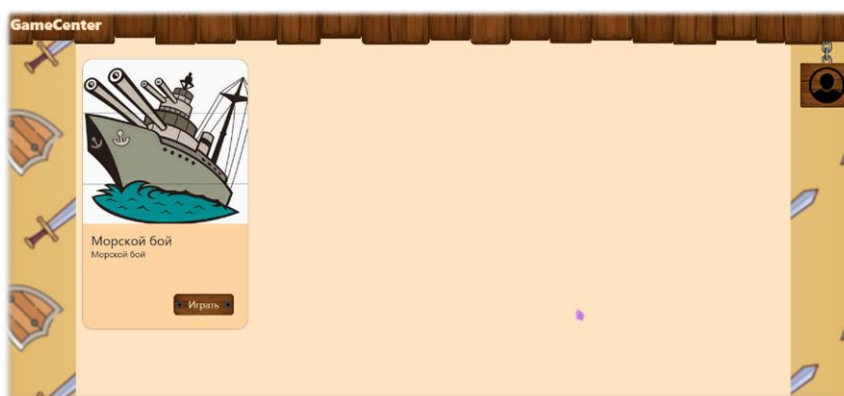


Рис. 5. Главная страница

Находясь на главной странице, пользователь может либо выбрать игру, чтобы поиграть, либо после авторизации перейти на страницу своего аккаунта, где можно будет открыть достижения и редактировать профиль (Изменить никнейм или аватар).

Чтобы изменить аватар необходимо нажать на свой аватар, после чего откроется страница с выбором аватара на замену.

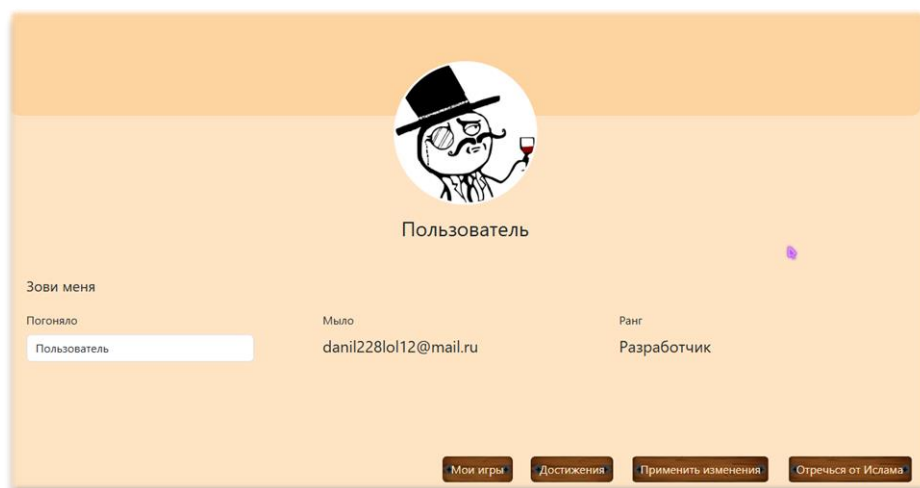


Рис. 6. Страница аккаунта

II. РАЗРАБОТКА ПРОГРАММНОГО ПРОДУКТА

Прежде чем приступать к разработке, стоит отметить, что необходимо иметь установленную программную платформу Node.js версии 16 или выше, а также какую-либо среду разработки, например, Visual Studio Code (используется в рисунках ниже).

2.1. Архитектура проекта и файлов

Для определения директории проекта нужно открыть командную строку (терминал) в папке проекта (используя VS Code) и использовать следующую команду:

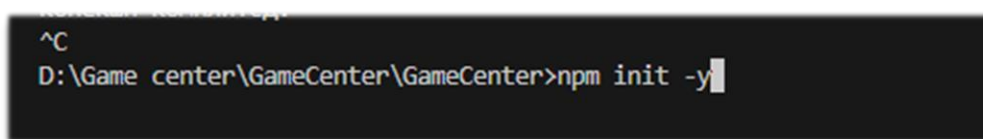


Рис. 7. Архитектура проекта

Архитектура директорий является важной составляющей любого проекта. Правильно организованная структура папок помогает разработчикам лучше ориентироваться в коде, облегчает процесс разработки и совместной работы в команде.

Четкая и логичная иерархия директорий позволяет быстро находить необходимые файлы и минимизирует риск путаницы. Это особенно важно в больших проектах, где количество компонентов может быть значительным.

Кроме того, хорошая архитектура директорий поддерживает масштабируемость проекта. Она закладывает основу для добавления новых функций, что делает код более управляемым и поддерживаемым. Соблюдение принципов организации файловой структуры также способствует лучшему восприятию проекта другими разработчиками, что особенно важно при передаче или переделке проекта.

В целом, архитектура директорий — это не просто косметическая деталь. Это основа, на которой строится весь проект, определяющая его доступность, удобство и долговечность.

						КР-09.02.07-Ф-30-25ПЗ	Лист
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		33

Моя архитектура выглядит примерно так

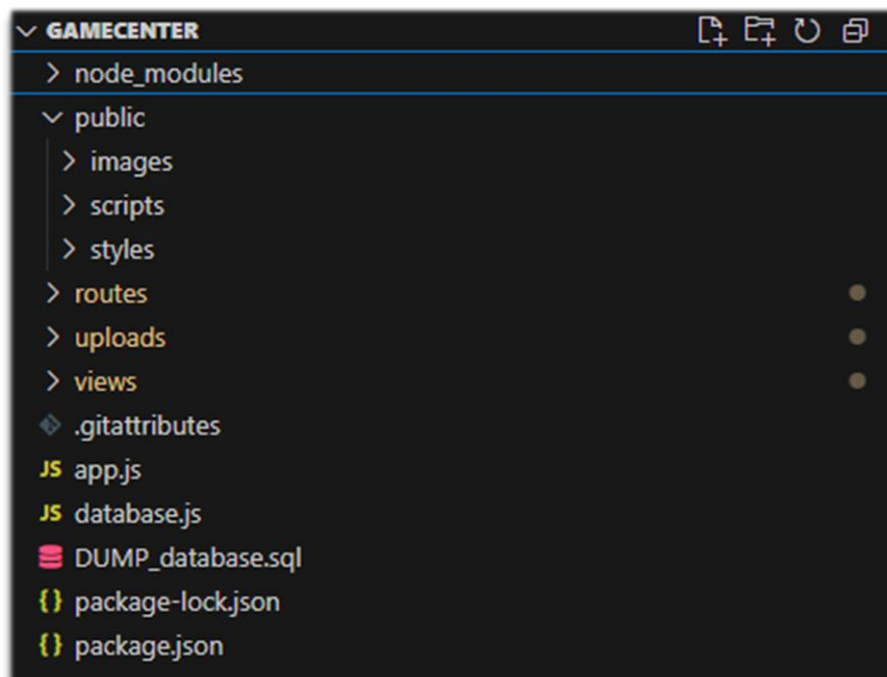


Рис. 8. Архитектура директории

После инициализации проекта необходимо установить нужные для работы фреймворки, для этого в консоли пишем «npm install (фреймворк который устанавливаем)»

Что нужно установить:

- express.js – основной фреймворк.
- express-session – дополнение к express для сохранения данных в сессию
- ejs – движок для динамической верстки html страниц
- bcrypt – фреймворк для «хеширования» паролей
- multer - фреймворк для упрощения работы с файловыми форматами
- fs – фреймворк для работы с файлами директории через код
- mysql2 – фреймворк для связи проекта с базой данных
- path – стандартный модуль, для работы с путями к файлам

Далее необходимо создать основной файл сервер (в моем случае он называется app.js) и заполнить его:

- Сначала надо подключить необходимые для работы фреймворки (для главного файла это express, express-session и path)

```
const express = require('express');
const session = require('express-session');
const path = require('path');
```

- Также вверху необходимо прописать путь к основному файлу маршрутизации и имени express модуля

```
const app = express();
const indexRouter = require('./routes/index');
```

- Далее устанавливаем, как движок представлений – ejs и указываем, в каких папках будут располагаться представления.

Также указываем папки для «статике»

```
app.set('view engine', 'ejs');
app.set('views', [
  path.join(__dirname, 'views'),
  path.join(__dirname, 'uploads')
]);
app.use(express.static('public'));
app.use(express.static('uploads'));
```

- Устанавливаем стандартные данные для session
- Указываем пути для основной и динамически подключающейся маршрутизации

```
app.use('/', indexRouter);
app.use('/game/:id', (req, res, next) => {...});
```

- Ну и наконец устанавливаем прослушивание для порта

```
app.listen(PORT, () => {
  console.log(`Server is running on port ${PORT}`);
});
```

Следующий блок будет про файл database.js. Я решил в своем проекте что вынесу функции обращения к базе данных MySQL в отдельный файл для оптимизации и уменьшения кода в основном файле маршрутизации.

Для удобства объяснения я разделил этот файл на три части:

						КР-09.02.07-Ф-30-25ПЗ	Лист
							35
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

- **HEADER** – подключение фреймворка mysql2(для работы с базой данных MySQL) и создание соединения (connection) с базой моей базой данных.

```
const mysql = require('mysql2');
const connection = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: 'root',
  database: 'GameCenter'
});
```

- **BODY** – тут будут располагаться все функции обращения к базе данных. Пример:

```
function findUserByUsername(email, callback) {
  const query = 'SELECT * FROM customers WHERE customerEmail = ?';
  connection.query(query, [email], callback);
}
```

- **FOOTER** – В этой част располагается модуль экспорта функций и соединения.

```
module.exports = {
  connection,
  «остальные функции»
};
```

Далее надо создать «Сердце маршрутизации» - файл route.js, который будет располагаться по пути ./routes/route.js . В этом файле будут прописаны все основные (не динамические) маршруты на сервере и функции для их обработки. Структура этого файла очень похожа на структуру файла database.js, только вот в HEADER подключается куда больше фреймворков, а также прописывается путь к уже созданному ранее файлу database.js:

```
const express = require('express');
const router = express.Router();
const connection = require('../database');
const bcrypt = require('bcrypt');
const multer = require('multer');
const fs = require('fs');
const path = require('path');
```

А снизу в FOOTER экспортируется только модуль router

						КР-09.02.07-Ф-30-25ПЗ	Лист
							36
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

```
module.exports = router;
```

Так называемое BODY файла router.js содержит все маршрут на сервере, основные из них:

- `router.get('/', (req, res) => {...});`
Главная страница
- `router.get('/login', (req, res) => {...});`
- `router.post('/login', (req, res) => {...});`
Переход на страницу авторизации и отправка формы авторизации.

Форма авторизации принимает значения из двух полей почты и пароля, после чего сверяет их с базой данных и отдает результат, в случае успеха пользователь переходит на главную страницу, а в случае неудачи пользователя перенаправит обратно на страницу авторизации и напишет, под формой, в чем ошибка.

- `router.get('/register', (req, res) => {...});`
- `router.post('/register', (req, res) => {...});`
Переход на страницу регистрации и отправка формы регистрации.

Форма регистрации работает аналогично авторизации, но принимает значения из четырех полей логина, почты, пароля и подтверждения пароля, после чего проверяет валидность данных и узнает нет ли такого пользователя в базе данных, если нет, пользователь добавится в базу данных и перенаправит на главную страницу, а если такой пользователь есть вернет на страницу регистрации и под формой напишет что такой пользователь есть.

- `router.get('/acc_page', (req, res) => {...});`
Направит авторизованного пользователя на страницу его аккаунта, где он сможет открыть свои достижения, изменить аватар и никнейм, выйти из аккаунта, а также если у пользователя ранг разработчик он сможет открыть «свои игры»

						КР-09.02.07-Ф-30-25ПЗ	Лист
							37
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

- `router.get('/logout', (req, res) => {...});`
Разрывает сессию пользователя и перенаправляет на главную страницу
- `router.get('/game/:id', (req, res) => {...});`
Переход на страницу игры. «id» передает идентификатор выбранной игры.
- `router.get('/add', (req, res) => {...});`
- `router.post('/add', (req, res) => {...});`
Форма добавления игры и маршрут для перехода на страницу добавления игры. Форма принимает значения заполненных поле и проверяет их на валидность после чего, если нет ошибок, создает папку с id пользователя и добавляет туда файлы новой игры, также эта форма создает запись в базу данных об этой игре.
- `router.get('/edit/:id', (req, res) => {...});`
- `router.post('/edit/:id', (req, res) => {...});`
Переход на страницу изменения игры и форма изменения данных игры. Работает аналогично форме добавления игры, только показывает какие файлы уже есть и также реализует функцию их замены на новые, посредством удаления старых и установки новых.
- `router.post('/delete/:id', (req, res) => {...});`
Маршрут для удаления игры по ее id
- `router.get('/achievements', (req, res) => {...});`
Переход на страницу достижений, где пользователь может посмотреть все достижения на сайте и уже достигнутые им.
- `router.get('/logoutandchange', (req, res) => {...});`
Реализует функционал сразу для нескольких кнопок (Мои игры, Применить изменения, Выйти, Достижения).
- И другие маршруты
-

						КР-09.02.07-Ф-30-25ПЗ	Лист
							38
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

2.2. Визуальная составляющая сайта

Теперь у нас есть основа, так сказать «база» для проекта, но нет еще одной ключевой детали – ejs представлений.

Они есть «визуал» нашего проекта, то с чем будет взаимодействовать непосредственно пользователь.

Вот все представления что используются на сайте(кроме динамически подключающихся):

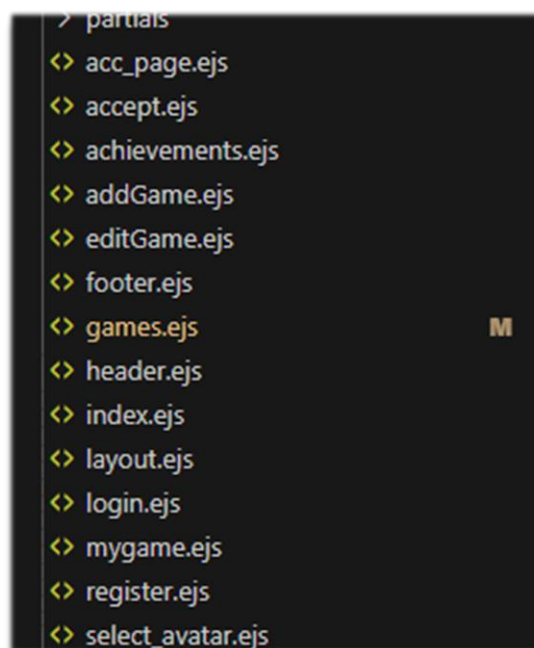


Рис. 9. EJS представления

Основным представлением является «layout.ejs», так скажем «основной скелет». В нем прописывается подключение скриптов, стилей и динамически подключаемого ejs представления.

```
<%- include('header'); %>
```

```
<%- include('partials/flash'); %>
```

```
<%- include(body); %>
```

```
<%- include('footer'); %>
```

- Header – это представление nav bara, навигации сайта.

- Flash – представление созданное для одной лишь функции отображения ошибок оно принимает две переменные `global_success` и `global_error` в которые можно передавать текст ошибки и успеха.
- Body – переменная в которую передается `ejs` представление которое нужно подгрузить в тело сайта
- Footer – представление «подвала» сайта

Остальные представления нужны для перемещения их в переменную `body`. Немного о каждом из этих представлений:

- `acc_page.ejs` – Страница аккаунта.
- `achievements.ejs` – Страница достижений.
- `addGame.ejs` – Страница добавления игры.
- `editGame.ejs` – Страница изменения игры.
- `game.ejs` – представление Главной страницы с играми.
- `login.ejs` – Страница авторизации.
- `register.ejs` – Страница регистрации.
- `select_avatar.ejs` – Страница выбора «аватарки».

Теперь осталась только «статика» проекта, которая находится в корневой папке проекта:

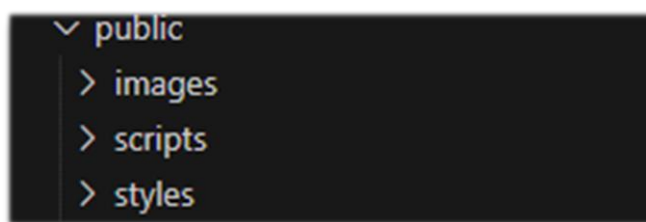


Рис. 10. «Статика» проекта

В папке `images` хранятся все изображения придающие «визуал» сайту.

В папке `scripts` хранятся скрипты сайта.

В папке `styles` хранится файл стилей.

						КР-09.02.07-Ф-30-25ПЗ	Лист 40
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

Для создания изображения на сайте я использовал web-сервис Figma, все стандартные изображения на сайте были спроектированы ней.

Разрабатывая дизайн сайт мне хотелось сделать так, чтобы он выделялся своим оформлением. Пусть это будет не «шедевр», но уникально творение, в связи с чем на разработку дизайна я потратил минимум 8 часов в Figma перебирая разные варианты дизайна пока не пришел к текущему.

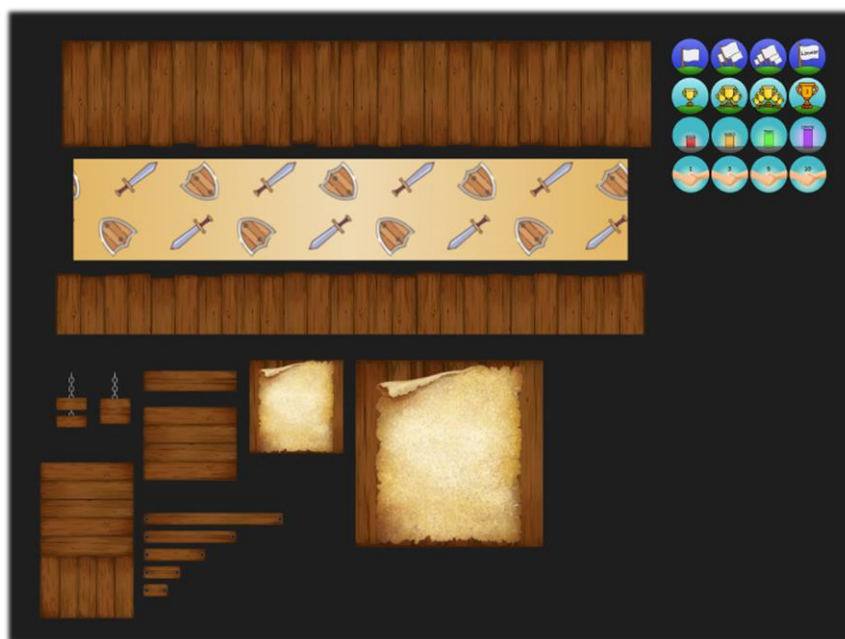


Рис. 10. Figma

Также хочется акцентировать внимание на изображениях достижений, главной идеей было сделать их в общем стиле и наглядно отображающими, что нужно сделать чтобы получить достижение.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была разработана архитектура и реализовано web-приложение для размещения и запуска браузерных игр с возможностью загрузки пользовательского контента. В процессе работы были решены следующие задачи:

Анализ существующих решений – изучены популярные платформы для браузерных игр.

Проектирование системы – разработана ER-диаграмма базы данных, выбраны технологии (Node.js, Express.js, MySQL, EJS, Bootstrap 5).

Реализация функционала – созданы:

Система регистрации и авторизации пользователей.

Механизм загрузки и запуска игр.

Личный кабинет с возможностью редактирования профиля и управления играми.

Система достижений для повышения вовлечённости пользователей.

Результаты работы:

Создано полноценное веб-приложение с интуитивно понятным интерфейсом.

Реализована возможность добавления пользовательских игр, что отличает проект от большинства аналогов.

Обеспечена безопасность данных (хеширование паролей, защита от SQL-инъекций).

Перспективы развития проекта:

Внедрение мультиплеерных режимов.

Оптимизация производительности для работы с большим количеством пользователей.

Разработанное приложение может быть использовано как игроками для доступа к разнообразным браузерным играм, так и разработчиками для публикации своих проектов.

						КР-09.02.07-Ф-30-25ПЗ	Лист
							42
Изм.	Кол.уч.	Лист	№ докум.	Подпись	Дата		

СПИСОК ЛИТЕРАТУРЫ

1. MDN Web Docs – документация по HTML5, CSS3, JavaScript. [Электронный ресурс]. URL: <https://developer.mozilla.org/>
2. Node.js Official Documentation – руководство по Node.js. [Электронный ресурс]. URL: <https://nodejs.org/en/docs/>
3. Express.js Documentation – официальная документация фреймворка Express. [Электронный ресурс]. URL: <https://expressjs.com/>
4. MySQL Documentation – руководство по работе с MySQL. [Электронный ресурс]. URL: <https://dev.mysql.com/doc/>
5. Bootstrap 5 Documentation – руководство по использованию Bootstrap. [Электронный ресурс]. URL: <https://getbootstrap.com/docs/5.0/getting-started/introduction/>
6. EJS Documentation – руководство по шаблонизатору EJS. [Электронный ресурс]. URL: <https://ejs.co/>
7. Figma Official Guide – документация по дизайну интерфейсов. [Электронный ресурс]. URL: <https://www.figma.com/resources/docs/>
8. Дейтел, П., Дейтел, Х. «Как программировать на JavaScript». – М.: Вильямс, 2020. – 1088 с.
9. Флэнаган, Д. «JavaScript. Подробное руководство». – СПб.: Символ-Плюс, 2021. – 1080 с.
10. Васильев, А.Н. «Node.js. Разработка серверных веб-приложений». – М.: Питер, 2022. – 352 с.
11. Кириченко, А.В. «Веб-разработка на JavaScript и Node.js». – СПб.: БХВ-Петербург, 2023. – 496 с.
12. Oracle Corporation «MySQL 8.0 Reference Manual». – 2023. [Электронный ресурс]. URL: <https://dev.mysql.com/doc/refman/8.0/en/>
13. Таненбаум, Э., Вудхалл, А. «Операционные системы. Разработка и реализация». – М.: Питер, 2019. – 704 с.

						КР-09.02.07-Ф-30-25ПЗ	Лист
							43
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

14. Столлинс, В. «Современные операционные системы». – М.: Вильямс, 2020. – 1120 с.
15. Кузнецов, М.В., Симдянов, И.В. «РНР 8. Объекты, шаблоны и методики программирования». – СПб.: БХВ-Петербург, 2022. – 768 с.
16. Фаулер, М. «Рефакторинг. Улучшение существующего кода». – М.: Символ-Плюс, 2019. – 448 с.
17. Гамма, Э., Хелм, Р., Джонсон, Р., Влиссидес, Дж. «Приемы объектно-ориентированного проектирования. Паттерны проектирования». – М.: Питер, 2021. – 496 с.
18. Макконнелл, С. «Совершенный код». – М.: Русская редакция, 2020. – 896 с.
19. W3C Standards – официальные стандарты веб-технологий. [Электронный ресурс]. URL: <https://www.w3.org/>
20. GitHub Documentation – руководство по работе с Git и GitHub. [Электронный ресурс]. URL: <https://docs.github.com/>

						КР-09.02.07-Ф-30-25ПЗ	Лист
							44
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		